



Actividad:

Actividad 1: "Del Cuello de Botella a la Concurrencia Multihilo"

Asignatura:

Programación aplicaciones de red

Alumno:

- Buenrostro Avila Abiel Gustavo
- Ramírez Rendon Naomi Elena
- Gómez González Victor Andres

Programa Educativo: **IdelO**

Presentado a:

PROF. Joyner Mazariegos Rabanales

Limitaciones del Servidor TCP Secuencial frente al Concurrente con Threading

1. Objetivo General

Demostrar empíricamente las limitaciones de un servidor TCP de un solo hilo (secuencial) al atender múltiples clientes simultáneos, compararlo con un servidor concurrente basado en `threading` de Python, y analizar los riesgos de condición de carrera que introduce la concurrencia.

2. Fase 1 — Cuello de Botella Secuencial

2.1 Explicación del Código

```
# fase1_servidor_secuencial.py (fragmento principal)
while True:
    conn, addr = srv.accept()          # (1) Bloquea hasta recibir conexión
    print(f"[{timestamp()}] Conexión aceptada de {addr}")
    with conn:
        time.sleep(10)                 # (2) Simula procesamiento pesado
        conn.sendall(b"OK - Procesado\n") # (3) Responde y libera
```

Bloque	Rol
<code>srv.accept()</code>	Extrae una conexión de la cola TCP del kernel y devuelve el socket de la conversación. Mientras no retorna, el proceso no puede hacer nada más.
<code>time.sleep(10)</code>	Simula una tarea costosa (consulta a BD, cómputo). Durante estos 10 s el proceso duerme; ningún otro cliente es atendido a nivel de aplicación.
<code>conn.sendall(...)</code>	Envía la respuesta y libera el socket; recién aquí el bucle regresa a <code>accept()</code> .

2.2 Procedimiento de Prueba

Se usaron **tres consolas** de PowerShell:

1. Consola A — Servidor:

```
python fase1_servidor_secuencial.py
```

2. Consola B — Cliente 1 (lanzado primero):

```
python cliente_prueba.py
```

3. Consola C — Cliente 2 (lanzado ~1 s después):

```
python cliente_prueba.py
```

2.3 Evidencias

The screenshot shows two Windows PowerShell windows. The left window shows the execution of `python .\cliente_prueba.py`, which connects to 127.0.0.1:8000, sends a message, and receives a response. The right window shows the execution of `python .\ase1_servidor_secuencial.py`, which listens on 127.0.0.1:8000 and processes the client's request.

```
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\cliente_prueba.py
[21:32:47.126] Conectando a 127.0.0.1:8000 ...
[21:32:47.127] Conexión establecida. Enviando mensaje ...
[21:32:57.128] Respuesta recibida: 'OK - Procesado'
[21:32:57.129] Latencia total: 10.00 s
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> |

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\cliente_prueba.py
[21:32:51.941] Conectando a 127.0.0.1:8000 ...
[21:32:51.942] Conexión establecida. Enviando mensaje ...
[21:33:07.131] Respuesta recibida: 'OK - Procesado'
[21:33:07.131] Latencia total: 15.19 s
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes>

PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\ase1_servidor_secuencial.py
[21:32:40] Servidor SECUENCIAL escuchando en 127.0.0.1:8000
[21:32:40] Solo se atiende UN cliente a la vez.

[21:32:47] Conexión aceptada de 127.0.0.1:51961
[21:32:47] Iniciando procesamiento (sleep 10s) ...
[21:32:57] Respuesta enviada a 127.0.0.1:51961. Conexión cerrada.

[21:32:57] Conexión aceptada de 127.0.0.1:51962
[21:32:57] Iniciando procesamiento (sleep 10s) ...
[21:33:07] Respuesta enviada a 127.0.0.1:51962. Conexión cerrada.
```

The screenshot shows a Telnet window and a Windows PowerShell window. The Telnet window shows the connection to 127.0.0.1, the response 'OK - Procesado', and the connection loss. The PowerShell window shows the execution of the server script, which listens on 127.0.0.1:8000 and processes the client's request.

```
Telnet 127.0.0.1
OK - Procesado
Connection to host lost.

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\ase1_servidor_secuencial.py
[22:20:07] Servidor SECUENCIAL escuchando en 127.0.0.1:8000
[22:20:07] Solo se atiende UN cliente a la vez.

[22:20:14] Conexión aceptada de 127.0.0.1:53615
[22:20:14] Iniciando procesamiento (sleep 10s) ...
[22:20:24] Respuesta enviada a 127.0.0.1:53615. Conexión cerrada.

[22:20:24] Conexión aceptada de 127.0.0.1:53619
[22:20:24] Iniciando procesamiento (sleep 10s) ...
[22:20:34] Respuesta enviada a 127.0.0.1:53619. Conexión cerrada.

[22:20:40] Conexión aceptada de 127.0.0.1:53627
[22:20:40] Iniciando procesamiento (sleep 10s) ...
[22:20:50] Respuesta enviada a 127.0.0.1:53627. Conexión cerrada.

[22:20:50] Conexión aceptada de 127.0.0.1:53629
[22:20:50] Iniciando procesamiento (sleep 10s) ...
[22:21:00] Respuesta enviada a 127.0.0.1:53629. Conexión cerrada.
```

2.4 Análisis del Bloqueo

Cuando el **Cliente 1** se conecta, el servidor lo acepta e inmediatamente entra en `sleep(10)`. Durante ese tiempo el **Cliente 2** intenta conectarse; el **kernel** completa el *TCP three-way handshake* (SYN → SYN-ACK → ACK) y almacena la conexión en la **cola de backlog** del socket. Sin embargo, la **aplicación** no llama a `accept()` hasta que termina de atender al Cliente 1.

Consecuencia observable:

Cliente	Latencia medida	Razón
Cliente 1	≈ 10 s	Atendido directamente
Cliente 2	≈ 20 s	Espera 10 s en backlog + 10 s de procesamiento propio

El cuello de botella es estructural: un único hilo de ejecución serializa todas las conexiones. Con N clientes simultáneos, el cliente N-ésimo esperaría $\approx N \times 10$ s, degradación **O(n)**.

3. Fase 2 — Servidor Concurrente con Threading

3.1 Cambio Arquitectónico

El modelo **thread-per-client** separa el rol del hilo principal del procesamiento por cliente:

```
# fase2_servidor_concurrente.py (fragmento principal)
def manejar_cliente(conn, addr):
    hilo = threading.current_thread().name
    with conn:
        time.sleep(10)
        conn.sendall(b"OK - Procesado\n")

while True:
    conn, addr = srv.accept()           # (1) Solo acepta
    t = threading.Thread(
        target=manejar_cliente,
        args=(conn, addr),
        daemon=True,
    )
    t.start()                           # (2) Delega inmediatamente
```

El hilo principal regresa a `accept()` en microsegundos; cada cliente vive en su propio hilo con su propio stack de ejecución. Los `sleep(10)` corren en paralelo porque el **GIL** (Global Interpreter Lock) de CPython se libera durante operaciones de I/O y `time.sleep`, permitiendo verdadero solapamiento de tiempos de espera.

3.2 Procedimiento de Prueba

Idéntico al de la Fase 1: tres consolas, mismos comandos.

3.3 Evidencias

```

PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\cliente_prueba.py
[22:04:21.973] Conectando a 127.0.0.1:8080 ...
[22:04:21.974] Conexión establecida. Enviando mensaje ...
[22:04:31.976] Respuesta recibida: 'OK - Procesado'
[22:04:31.977] Latencia total: 10.00 s
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes>

PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\ase2_servidor_concurrente.py
[22:04:18] Servidor CONCURRENTE escuchando en 127.0.0.1:8080
[22:04:18] Modelo: un hilo por cliente (thread-per-client).
[22:04:21] [Thread-1 (manejar_cliente)] Atendiendo a 127.0.0.1:51360
[22:04:21] [MainThread] Hilo lanzado: Thread-1 (manejar_cliente) para 127.0.0.1:51360
[22:04:21] [Thread-1 (manejar_cliente)] Iniciando procesamiento (sleep 10s)
...
[22:04:24] [Thread-2 (manejar_cliente)] Atendiendo a 127.0.0.1:51362
[22:04:24] [MainThread] Hilo lanzado: Thread-2 (manejar_cliente) para 127.0.0.1:51362
[22:04:24] [Thread-2 (manejar_cliente)] Iniciando procesamiento (sleep 10s)
...
[22:04:31] [Thread-1 (manejar_cliente)] Respuesta enviada a 127.0.0.1:51360. Conexión cerrada.
[22:04:34] [Thread-2 (manejar_cliente)] Respuesta enviada a 127.0.0.1:51362. Conexión cerrada.
  
```

```

OK - Procesado

Connection to host lost.
PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes>

PS C:\Users\vican\OneDrive\Escritorio\Claude Code\proyecto-redes> python .\ase2_servidor_concurrente.py
[22:22:51] Servidor CONCURRENTE escuchando en 127.0.0.1:8080
[22:22:51] Modelo: un hilo por cliente (thread-per-client).
[22:22:54] [Thread-1 (manejar_cliente)] Atendiendo a 127.0.0.1:63038
[22:22:54] [MainThread] Hilo lanzado: Thread-1 (manejar_cliente) para 127.0.0.1:63038
[22:22:54] [Thread-1 (manejar_cliente)] Iniciando procesamiento (sleep 10s)
...
[22:22:55] [Thread-2 (manejar_cliente)] Atendiendo a 127.0.0.1:63039
[22:22:55] [MainThread] Hilo lanzado: Thread-2 (manejar_cliente) para 127.0.0.1:63039
[22:22:55] [Thread-2 (manejar_cliente)] Iniciando procesamiento (sleep 10s)
...
[22:23:04] [Thread-1 (manejar_cliente)] Respuesta enviada a 127.0.0.1:63038. Conexión cerrada.
[22:23:05] [Thread-2 (manejar_cliente)] Respuesta enviada a 127.0.0.1:63039. Conexión cerrada.
  
```

3.4 Análisis

Ambos clientes terminan en ≈ 10 s porque sus `sleep` ocurren **simultáneamente** en hilos distintos. El log del servidor muestra nombres como `Thread-1` y `Thread-2`, evidenciando que cada conexión tiene su propio contexto de ejecución.

Cliente	Fase 1	Fase 2
Cliente 1	≈ 10 s	≈ 10 s
Cliente 2	≈ 20 s	≈ 10 s
Mejora	—	50 % menos latencia para el segundo cliente

Con N clientes simultáneos, todos terminan en ≈ 10 s (la duración del procesamiento), independientemente de N, mientras los recursos del sistema lo permitan.

4. Fase 3 — Reflexión Analítica: Condición de Carrera

4.1 Definición

Una **condición de carrera** (*race condition*) ocurre cuando dos o más hilos acceden a un recurso compartido y el resultado final depende del **orden de planificación** del sistema operativo, que no es determinístico.

4.2 Ejemplo Concreto

Supón un sistema de ventas donde 100 hilos procesan transacciones simultáneas:

```
ventas = 0 # variable compartida

def procesar_venta(monto):
    global ventas
    ventas += monto # NO es atómica: read → modify → write
```

La operación `ventas += monto` se descompone internamente en tres pasos:

1. **Leer** el valor actual de `ventas` (p. ej. `500`).
2. **Sumar** `monto` (p. ej. `+ 100 → 600`).
3. **Escribir** `600` de vuelta en `ventas`.

Si dos hilos ejecutan el paso 1 antes de que alguno ejecute el paso 3, ambos leen `500`, calculan `600`, y escriben `600`. Se **pierde** una actualización: el resultado es `600` en lugar de `700`.

4.3 Solución: Exclusión Mutua con `threading.Lock`

```
import threading

ventas = 0
lock = threading.Lock()

def procesar_venta(monto):
    global ventas
    with lock:          # solo un hilo entra a este bloque a la vez
        ventas += monto
```

El `with lock:` garantiza que el bloque read-modify-write sea **atómico** desde el punto de vista del programa: ningún otro hilo puede interrumpirlo.

4.4 Alternativas y Trade-offs

Mecanismo	Uso ideal	Limitación
<code>threading.Lock</code>	Sección crítica simple	Contención si muchos hilos compiten
<code>threading.RLock</code>	Mismo hilo necesita re-adquirir el lock	Más overhead que <code>Lock</code>
<code>threading.Semaphore(n)</code>	Limitar acceso a N hilos simultáneos	No garantiza orden
<code>queue.Queue</code>	Patrón productor-consumidor	Desacopla productores de consumidores
Operaciones atómicas	Contadores simples (<code>collections.Counter</code>)	Limitado a tipos básicos

Trade-off clave: cuanto más **grueso** el lock (proteger bloques grandes), más simple el código pero mayor la **contención** (los hilos esperan más). La estrategia óptima es usar locks de **granularidad fina**: proteger solo la sección crítica mínima necesaria.

5. Conclusiones

Gómez González Víctor Andres:

La Fase 1 demuestra que un servidor secuencial es incapaz de escalar; el bloqueo en `accept()` convierte cualquier latencia de procesamiento en un efecto dominó que degrada el servicio a $O(n)$. En sistemas de producción, esto se traduce en una saturación del backlog y en una experiencia de usuario inaceptable, ya que la disponibilidad depende de que no existan otros clientes activos, lo cual es inviable en entornos reales.

Buenrostro Avila Abiel Gustavo:

El paso al modelo concurrente en la Fase 2 reduce drásticamente la latencia al permitir que los tiempos de espera de I/O se solapen. Sin embargo, esto traslada la carga al programador, quien debe gestionar la seguridad de los hilos (thread-safety). Aunque threading es ideal para tareas bloqueantes como esta, para procesos con alta carga de CPU sería preferible usar multiprocessing para evadir el GIL de Python.

Ramírez Rendon Naomi Elena:

La concurrencia introduce el riesgo de condiciones de carrera, lo que vuelve indispensables las herramientas de sincronización como `threading.Lock`. La clave es encontrar el equilibrio en la granularidad del bloqueo: proteger la integridad de los datos sin generar una contención que convierta el servidor nuevamente en algo prácticamente secuencial. La Fase 3 confirma que la atomicidad en variables compartidas es la base de un sistema distribuido confiable.

Anexo — Comandos para Reproducir

```
# Fase 1
python fase1_servidor_secuencial.py      # Consola A
python cliente_prueba.py                  # Consola B
python cliente_prueba.py                  # Consola C (casi simultáneo)

# Fase 2 (detener servidor anterior con Ctrl+C primero)
python fase2_servidor_concurrente.py     # Consola A
python cliente_prueba.py                  # Consola B
python cliente_prueba.py                  # Consola C (casi simultáneo)
```

Repositorio GitHub: [INSERTAR_URL]